

Glosario del orquestador

Los conceptos de IA que hay detrás de la aplicación, cada uno explicado en dos frases y anclado a la línea de código donde vive. Pensado para leerse de un tirón o para consultarse suelto.

A · Hablar con el modelo B · Convertirlo en agente C · Que no mienta D · Operarlo

Modelo de lenguaje LLM

Un programa que recibe texto y devuelve texto. No sabe nada del mundo que no le hayas contado en esa misma petición, y no recuerda la conversación anterior.

Aquí Nunca se le pide un precio ni un cálculo: solo que redacte o que decida sobre datos que ya ha calculado el código.

providers/llm_provider.py:128 `class LLMProvider`

Prompt de sistema system prompt

Las instrucciones permanentes que definen el papel del modelo y sus límites, separadas de la pregunta concreta de cada turno. Es donde se escriben las reglas que no deben negociarse.

Aquí Hay dos, uno por modo, y ambos empiezan por lo mismo: «no inventes precios, indicadores, fechas ni fuentes».

providers/llm_provider.py:59
`SYSTEM_PROMPT · :73 AGENT_SYSTEM_PROMPT`

Abstracción de proveedor

`provider abstraction`

Una interfaz común para que el resto del código no sepa qué modelo hay detrás. Cambiar de proveedor es cambiar una variable de entorno, no reescribir agentes.

Aquí Tres implementaciones: Claude, OpenRouter y un simulador local. Se elige con `LLM_PROVIDER`.

providers/anthropic_provider.py,
`openrouter_provider.py,`
`mock_llm_provider.py` · fábrica en **main.py:45**

Prompt por capas layered prompting

El mensaje de usuario no es una frase suelta: se ensambla con partes de origen distinto —tarea, instrucciones del agente, hechos, esquema de respuesta— y cada capa se puede auditar aparte.

Aquí Los datos van dentro de un bloque `<facts>` delimitado, para que quede claro qué es dato y qué es instrucción.

providers/llm_provider.py:209
`render_agent_prompt()`

Salida estructurada

`structured output` · `JSON Schema`

En lugar de prosa libre, se le exige al modelo un objeto JSON con campos y tipos fijos. Así otro programa puede leer la respuesta sin adivinar.

AQUÍ Cada agente declara su esquema —postura, confianza, evidencia, resumen—, y con Claude se impone en la propia llamada a la API.

`llm_provider.py:37` `EXPLANATION_SCHEMA` ·
`technical_agent.py:41` `TECH_SCHEMA` ·
`anthropic_provider.py:111`

Ventana de contexto `context window`

Todo lo que el modelo puede «ver» en una llamada: el sistema, el historial de mensajes y los resultados de herramientas. Fuera de ahí, para él no existe.

AQUÍ Cada agente construye su propia lista de mensajes desde cero y la tira al acabar. No hay un contexto global compartido entre agentes.

`providers/llm_provider.py:108`
`ChatMessage` · `base_agent.py:269` lista
`messages`

Tokens `tokens`

Los trozos en que el modelo parte el texto, y la unidad en que se factura: tantos de entrada, tantos de salida, a un precio distinto cada uno.

AQUÍ Cada respuesta trae su recuento; si el proveedor no informa del coste, se estima con una tabla de tarifas por millón de tokens.

`llm_provider.py:172` `with_cost()` ·
`anthropic_provider.py:32`
`ANTHROPIC_PRICES`

Esfuerzo y límite de salida

`effort` · `max_tokens`

Dos mandos: cuánto debe pensar el modelo antes de responder, y cuánto texto se le permite generar. Ambos cambian el coste y la latencia.

AQUÍ Esfuerzo bajo por defecto —las tareas son cortas—, y si una respuesta se trunca por el límite se trata como fallo, no como respuesta válida.

`config.py:48` `anthropic_effort` ·
`anthropic_provider.py:86` `_check_stop()`

Herramientas tool calling · function calling

Funciones normales de código que se le describen al modelo. Él no las ejecuta: responde «llama a esta con estos argumentos», y el programa decide si lo hace.

AQUÍ Cuatro en total: calcular indicadores, calcular métricas de riesgo, pedir más historial y retar al analista técnico.

llm_provider.py:94 ToolSpec (qué ve el modelo) · **base_agent.py:57** Tool (qué se ejecuta)

Bucle agéntico agent loop

Pedir → el modelo llama a una herramienta → ejecutarla → devolverle el resultado → repetir, hasta que entrega su respuesta final. Ese ciclo es lo que hace «agente» a un LLM.

AQUÍ Con tope de ocho vueltas. Si el modelo no cierra, se considera fallo y el agente cae a las reglas.

base_agent.py:250 run_agent_loop() · **config.py:79** agent_max_steps

Sistema multiagente multi-agent system

Varios agentes con una especialidad estrecha cada uno, en lugar de un único modelo al que se le pide todo. Más fácil de auditar, y cada pieza se puede mejorar por separado.

AQUÍ Cinco analistas y un coordinador, cada uno en su archivo, con una responsabilidad que no se solapa con las demás.

orchestrator.py:74 registro de agentes · **backend/app/agents/**

Paso de mensajes tipado

typed message passing

Los agentes no se llaman como funciones: se mandan mensajes con un tipo declarado — petición, resultado, opinión, reto, error—. El tipo dice qué se espera de vuelta.

AQUÍ Ocho tipos, y todo mensaje pasa por un único punto de entrega que lo publica antes de encaminarlo. Nada viaja sin dejar rastro.

models.py:49 MessageType · **orchestrator.py:84** deliver()

Orquestación centralizada

`central orchestration`

Un coordinador conoce el orden del proceso; los agentes solo conocen su tarea. Evita que todos tengan que saber de todos.

Aquí El orquestador fija la secuencia y lanza technical y risk en paralelo, pero no opina sobre el contenido.

orchestrator.py:168 `analyze_symbol()`

Comunicación directa `peer-to-peer`

Un agente puede dirigirse a otro sin pasar por el coordinador, cuando descubre que necesita algo que no estaba en el guion.

Aquí Dos casos: risk pidiendo más historial a market_data, y skeptic retando a technical. Ambos aparecen en la traza como mensajes normales.

risk_agent.py:124 `_request_history()` ·
skeptic_agent.py:100 `_challenge()`

Paso por referencia `shared context`

Los datos voluminosos no se copian dentro de los mensajes: se guardan una vez y los mensajes llevan solo una etiqueta que apunta a ellos.

Aquí La serie de precios vive en un diccionario compartido y viaja como `data_ref`, así que la traza sigue siendo legible.

base_agent.py:43 `AgentContext.data` ·
market_data_agent.py:46

Debate y crítica `adversarial review`

Un agente cuyo trabajo es contradecir a los demás, no repetir su análisis. Encontrar el fallo es más barato que evitarlo, y deja constancia de por qué la conclusión se sostiene.

Aquí skeptic busca incoherencias concretas y reta; technical responde «mantengo» o «concedo», y decision usa la postura *revisada*.

skeptic_agent.py:59
`find_counterarguments()` ·
decision_agent.py:57
`effective_technical()`

Alucinación hallucination

Cuando el modelo afirma algo con total naturalidad y resulta ser falso: un precio inventado, un indicador que nunca se calculó, un porcentaje que suena bien. No es un error del sistema, es cómo funciona por diseño.

AQUÍ Se asume que va a pasar. Todo el resto de esta familia existe para detectarlo antes de que llegue a la pantalla.

[services/validation.py](#) · docstring del módulo

Anclaje a hechos grounding

Obligar al modelo a apoyarse solo en datos que se le han dado, y *comprobarlo después*. Sin esa comprobación, «no inventes nada» es solo una recomendación.

AQUÍ Se extrae cada número del texto generado y se busca entre los hechos conocidos — admitiendo redondeos, la versión en porcentaje y periodos como 14, 20 o 50—. El que no aparece se marca.

[validation.py:78](#) `ungrounded_numbers()` · [:62](#) `_number_is_grounded()`

Hechos conocidos known facts

El universo contra el que se valida: los datos iniciales más todo lo que han devuelto las herramientas durante el bucle. Si el modelo obtuvo un dato, ese dato ya cuenta.

AQUÍ Se acumula vuelta a vuelta, y solo se añaden los resultados de herramientas que no fallaron.

[base_agent.py:270](#) `diccionario` `known`

Citación de evidencia evidence citation

No basta con la conclusión: el modelo debe listar en qué hecho concreto se apoya cada parte de su razonamiento, con el nombre exacto del dato.

AQUÍ Cada evidencia se comprueba contra los hechos conocidos; las que citan un hecho inexistente se descartan y quedan como aviso.

[validation.py:182](#) `bloque` `evidence` · [models.py](#) `Evidence`

Guardarraíles guardrails

Reglas deterministas que revisan la salida del modelo y la corrigen cuando cruza una línea. No mejoran el razonamiento: acotan el daño de un razonamiento malo.

AQUÍ Siete, con nombre propio. Por ejemplo, nunca se emite COMPRA sobre un activo de riesgo ALTO, diga lo que diga el modelo. Cada corrección se emite como evento visible.

base_agent.py:179 `guardrail()` ·
decision_agent.py:204 reglas G1-G4

Referencia determinista

deterministic baseline

Mantener una versión del cálculo hecha solo con reglas, aunque decida el modelo. Sirve de patrón de comparación y de red de rescate.

AQUÍ Si el modelo se aparta de la referencia sin evidencia suficiente, se corrige hacia ella; y la referencia se guarda junto a la opinión para poder comparar.

technical_agent.py:92 `rule_stance()` ·
risk_agent.py:77 `rule_risk_level()`

Degradación elegante graceful degradation

Que el fallo del componente menos fiable no tumbe el sistema: se pierde calidad, no servicio, y el usuario se entera de que se ha degradado.

AQUÍ Sin clave, sin red o con una respuesta inválida, cada agente cae al modo reglas y anota el motivo en su propia explicación.

technical_agent.py:168 `except LLMError`
· **base_agent.py:212**

Confianza acotada calibrated confidence

Un número de 0 a 1 que acompaña a cada opinión. Solo sirve si alguien comprueba que es coherente con la evidencia; si no, es decoración.

AQUÍ Se valida el rango, y se recorta cuando la evidencia no la sostiene: conceder un reto obliga a bajarla, y una discrepancia sin resolver la limita a 0,5.

validation.py:170 `acotado` ·
decision_agent.py:210 regla G3

Observabilidad `tracing`

Emitir un registro de todo lo que ocurre por dentro —quién empezó, qué se envió, qué corrigió una regla— en lugar de mostrar solo el resultado.

Aquí Quince tipos de evento numerados en orden, incluidos los incómodos: discrepancias, avisos de validación y guardarraíles aplicados.

`models.py:60` `EventType` · `events.py:26`
`EventBus.emit()`

Reproducibilidad `event sourcing` · `replay`

Si guardas la secuencia completa de eventos, puedes reconstruir cualquier ejecución pasada tal como sucedió, sin volver a llamar al modelo.

Aquí Cada ejecución se escribe en un JSON, y al abrir el WebSocket el servidor reenvía primero todo el historial y luego sigue en vivo.

`services/run_store.py:54` `save()` · `main.py:208` WebSocket con `replay`

Control de concurrencia

`throttling` · `rate limits`

Un proveedor de LLM tiene cuotas por minuto. Sin un tope propio, cinco símbolos en paralelo lanzan diez peticiones simultáneas y empiezan los errores.

Aquí Dos límites: tres símbolos a la vez y cuatro llamadas simultáneas al modelo, esta última envolviendo al proveedor sin que los agentes se enteren.

`llm_provider.py:149`
`ThrottledLLMProvider` · `config.py:89`
`max_parallel_symbols`

Simulador determinista `mock provider`

Una implementación falsa del proveedor que produce siempre lo mismo. Permite desarrollar y demostrar sin claves, sin red y sin gastar.

Aquí El simulador imita también el bucle de herramientas, así que la forma de la traza es idéntica a la de un modelo real.

`providers/mock_llm_provider.py:32` · `mock_market_data_provider.py:53`

Contabilidad de coste cost accounting

Atribuir el gasto a quién lo generó. En un sistema multiagente el coste se dispara sin avisar: cada vuelta del bucle es una llamada más.

Aquí Se agrega por agente a partir de los eventos y se muestra en vivo, con una etiqueta que dice si el coste es informado, estimado o desconocido.

services/costs.py:31 `summarize_costs()`
· **frontend/components/CostPanel.tsx**

Fallos del proveedor refusals · truncation

Una llamada a un LLM no falla solo por la red: puede agotarse la cuota, el modelo puede rechazar la petición, o la respuesta puede cortarse a mitad.

Aquí Cada caso se traduce a un error propio con mensaje legible, y un rechazo o un truncado se tratan como fallo —nunca como respuesta parcial válida.

anthropic_provider.py:65 `_create()` ·
:86 `_check_stop()`

La cadena completa, de una ojeada

Casi todos los conceptos anteriores encajan en una sola secuencia. Cada eslabón puede fallar, y para cada uno hay una respuesta prevista que no es «devolver algo inventado».



Si algo se rompe en cualquier punto, el agente cae al modo reglas, deja el motivo por escrito y la ejecución termina igualmente con un resultado por símbolo.

Documentos hermanos en `docs/`: **como funciona en 3 hojas.html** recorre la aplicación de punta a punta, y **conceptos ai-llm-y-preguntas.html** desarrolla estos mismos conceptos en versión larga, con ejemplos de prompts reales y preguntas frecuentes.